

LAPORAN TUGAS PRAKTIKUM 7
SISTEM OPERASI



Dosen Pengampu Mata Kuliah:
Triawan Adi Cahyanto, M.K

Disusun Oleh:

Nama : Nabilla Dwi Farasari
NIM : 2410651054
Kelas : C (Kamis)

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER

2025

Tugas 1: Analisis Memory Layout

A. Modifikasi Program

Tambahkan fitur-fitur berikut pada program:

1. Array Allocation: Buat array statis dan dinamis, tampilkan alamatnya
2. Multiple Functions: Buat 3 fungsi berbeda, tampilkan alamat setiap fungsi
3. Recursive Function: Buat fungsi rekursif dan tampilkan alamat stack di setiap level
4. String Literals: Tampilkan alamat string literals

```
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/background$ nano address_demo.c
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/background$ gcc address_demo.c
-o address_demo
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/background$ ./address_demo
1. Text Segment:
  main() address: 0x5c739704524b
  func1() address: 0x5c73970451c9
  func2() address: 0x5c73970451d4

2. Data Segment:
  static_array[0] address: 0x5c7397048010
  string_literal address: 0x5c7397046008

3. Heap:
  dynamic_array[0] address: 0x5c73a971d2a0
  malloc block 1: 0x5c73a971d2c0
  malloc block 2: 0x5c73a971d330

4. Stack (Recursive Calls):
  Level 0 local_var: 0x7fff5465d904
  Level 1 local_var: 0x7fff5465d8d4
  Level 2 local_var: 0x7fff5465d8a4
```

B. Analisis

1. Berapa jarak (dalam bytes) antara:

- Stack dan Heap?

Jawaban:

- Heap (dynamic_array[0]): 0x5c73a971d2a0
- Stack (Level 0 local_var): 0x7fff5465d904
- $0x7fff5465d904 - 0x5c73a971d2a0 = 39.082.775.545.444 \text{ bytes} \approx 37.27 \text{ TB}$

Jarak stack dan heap sangat jauh, stack berada di Alamat tinggi, heap di Alamat lebih rendah, ini menunjukkan pemisahan space yang jelas.

- Text segment dan Data segment?

Jawaban:

- Text (main): 0x5c739704524b
- Data (static_array[0]) : 0x5c7397048010
- $0x5c7397048010 - 0x5c739704524b = 11.717 \text{ bytes} \approx 11.4 \text{ KB}$

Data segment terletak setelah text segment, jarak kecil karena berada dalam satu executable image

- Alamat fungsi satu dengan lainnya?

Jawaban:

- func1(): 0x5c73970451c9
- func2(): 0x5c73970451d4
- main(): 0x5c73970451d4
- func2 – func1 = 0xB = 11 bytes
- main – faunc1 = 130 bytes

Fungsi disimpan berurutan di text segment, jarak kecil menunjukkan hasil kompilasi tanpa padding besar.

2. Jalankan program 5 kali, catat alamat-alamat yang muncul:

- Alamat mana yang selalu sama?

Jawaban:

- (Percobaan kedua)

```
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/background$ ./address_demo
1. Text Segment:
main() address: 0x5f6233fef24b
func1() address: 0x5f6233fef1c9
func2() address: 0x5f6233fef1d4

2. Data Segment:
static_array[0] address: 0x5f6233ff2010
string_literal address: 0x5f6233ff0008

3. Heap:
dynamic_array[0] address: 0x5f625d6b52a0
malloc block 1: 0x5f625d6b52c0
malloc block 2: 0x5f625d6b5330

4. Stack (Recursive Calls):
Level 0 local_var: 0x7ffd6b220894
Level 1 local_var: 0x7ffd6b220864
Level 2 local_var: 0x7ffd6b220834
```

- (Percobaan ketiga)

```
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/background$ ./address_demo
1. Text Segment:
main() address: 0x55f959bd424b
func1() address: 0x55f959bd41c9
func2() address: 0x55f959bd41d4

2. Data Segment:
static_array[0] address: 0x55f959bd7010
string_literal address: 0x55f959bd5008

3. Heap:
dynamic_array[0] address: 0x55f97c1342a0
malloc block 1: 0x55f97c1342c0
malloc block 2: 0x55f97c134330

4. Stack (Recursive Calls):
Level 0 local_var: 0x7ffc02fe6684
Level 1 local_var: 0x7ffc02fe6654
Level 2 local_var: 0x7ffc02fe6624
```

- (Percobaan keempat)

```
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/background$ ./address_demo
1. Text Segment:
  main() address: 0x60dcbde4624b
  func1() address: 0x60dcbde461c9
  func2() address: 0x60dcbde461d4

2. Data Segment:
  static_array[0] address: 0x60dcbde49010
  string_literal address: 0x60dcbde47008

3. Heap:
  dynamic_array[0] address: 0x60dcebaa42a0
  malloc block 1: 0x60dcebaa42c0
  malloc block 2: 0x60dcebaa4330

4. Stack (Recursive Calls):
  Level 0 local_var: 0x7ffc0fd08d14
  Level 1 local_var: 0x7ffc0fd08ce4
  Level 2 local_var: 0x7ffc0fd08cb4
```

- (Percobaan kelima)

```
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/background$ ./address_demo
1. Text Segment:
  main() address: 0x5903b26cf24b
  func1() address: 0x5903b26cf1c9
  func2() address: 0x5903b26cf1d4

2. Data Segment:
  static_array[0] address: 0x5903b26d2010
  string_literal address: 0x5903b26d0008

3. Heap:
  dynamic_array[0] address: 0x5903c94ec2a0
  malloc block 1: 0x5903c94ec2c0
  malloc block 2: 0x5903c94ec330

4. Stack (Recursive Calls):
  Level 0 local_var: 0x7ffed240e324
  Level 1 local_var: 0x7ffed240e2f4
  Level 2 local_var: 0x7ffed240e2c4
```

- Alamat mana yang berubah?
 - A. Text Segment
 - Run 1 main(): 0x5c739704524b
 - Run 2 main(): 0x5f6233fef24b
 - Run 3 main(): 0x55f959bd424b
 - Run 4 main(): 0x60dcbde4624b
 - Run 5 main(): 0x5903b26cf24b
 - B. Data Segment
 - Alamat static_array dan string_literal
 - C. Heap
 - Alamat dynamic_array dan hasil malloc
 - D. Stack
 - Run 1 Level 0: 0x7ff5465d904
 - Run 2 Level 0: 0x7ffd6b220894
 - Run 3 Level 0: 0x7ffc02fe6684

- Run 4 Level 0: 0x7ffc0fd08d14
- Run 5 Level 0: 0x7ffed240e324

- Jelaskan mengapa terjadi perbedaan tersebut (hint: ASLR - Address Space Layout Randomization)

Jawaban:

ASLR mengacak base address dari text, data, heap, dan stack setiap kali program dijalankan untuk meningkatkan keamanan sistem dan mencegah eksploitasi berbasis Alamat memori tetap. Meskipun Alamat absolut berubah, struktur dan jarak relative antar segment tetap konsisten.

3. Bandingkan dengan memory map dari /proc/[pid]/maps:

- Apakah alamat yang Anda dapatkan sesuai dengan range di memory map? Sebutkan permission (rwx) dari setiap segment

Jawaban:

- Alamat fungsi main(), func1(), dan func2() berada pada region executable program, yang termasuk text segment.
- Alamat static_array dan string_literal berada pada region data.
- Alamat hasil malloc (dynamic_array, block1, block2) berada pada region heap.
- Alamat variable local pada fungsi rekursif berada pada region stack.

Seluruh alamat yang ditampilkan program valid dan berada dalam rentang yang sesuai.

- Text Segmen = r-xp
- Data Segment = rw-p
- Heap = rw-p
- Stack = rw-p

Tugas 2: Implementasi Memory Allocator

Kembangkan program memory allocator dengan fitur tambahan dan lakukan analisis performa.

A. Implementasi Next-Fit Algorithm

Tambahkan algoritma Next-Fit pada program memory_allocator.c:

Next-Fit: Seperti First-Fit, tapi mulai pencarian dari posisi alokasi terakhir.

B. Fitur Fragmentasi

Tambahkan fungsi untuk menghitung fragmentasi:

```
void calculate_fragmentation() {
```

```
// Hitung:
// 1. Total free space
// 2. Largest free block
// 3. Number of free holes
// 4. External fragmentation ratio
}
```

```
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/contiguous-allocation$ ./memory_allocator

=== BENCHMARK MEMORY ALLOCATOR ===

--- FIRST FIT ---

Waktu Eksekusi: 3.00 microsecond
Total Free Space : 50
Largest Hole      : 30
Free Holes        : 2
External Frag %   : 40.00%
Failed Allocation: 1

--- BEST FIT ---

Waktu Eksekusi: 3.00 microsecond
Total Free Space : 50
Largest Hole      : 30
Free Holes        : 2
External Frag %   : 40.00%
Failed Allocation: 1

--- WORST FIT ---

Waktu Eksekusi: 2.00 microsecond
Total Free Space : 170
Largest Hole      : 100
Free Holes        : 2
External Frag %   : 41.18%
Failed Allocation: 1

--- NEXT FIT ---

Waktu Eksekusi: 2.00 microsecond
Total Free Space : 50
Largest Hole      : 30
Free Holes        : 2
External Frag %   : 40.00%
Failed Allocation: 1
```

C. Benchmark Testing

Buat test case untuk membandingkan 4 algoritma:

```
void benchmark_test() {
```

// Test case:

// 1. Allocate: P1(100), P2(200), P3(300), P4(250), P5(150) // 2. Deallocate: P2, P4

// 3. Allocate: P6(180), P7(220), P8(100)

// 4. Hitung waktu eksekusi dan fragmentasi untuk setiap algoritma }

```
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/contiguous-allocation$ gcc -o memory_allocator memory_allocator.c
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/contiguous-allocation$ ./memory_allocator

=== BENCHMARK MEMORY ALLOCATOR ===

--- FIRST FIT ---
=== STATUS MEMORI ===
Block      Size      Start      Status
-----
1          100        0          Allocated
6          180       100        Allocated
-1         20       280        Free
3          300       300        Allocated
7          220       600        Allocated
-1         30       820        Free
5          150       850        Allocated

Fragmentation Analysis:
Total Free Space      : 50
Largest Free Block    : 30
Number of Free Holes  : 2
External Fragmentation : 40.00%

--- BEST FIT ---
=== STATUS MEMORI ===
Block      Size      Start      Status
-----
1          100        0          Allocated
6          180       100        Allocated
-1         20       280        Free
3          300       300        Allocated
7          220       600        Allocated
-1         30       820        Free
5          150       850        Allocated

Fragmentation Analysis:
Total Free Space      : 50
Largest Free Block    : 30
Number of Free Holes  : 2
External Fragmentation : 40.00%
```

```

--- WORST FIT ---
=== STATUS MEMORI ===
Block      Size      Start      Status
-----
1          100        0          Allocated
8          100       100        Allocated
-1         100       200        Free
3          300       300        Allocated
6          180       600        Allocated
-1         70        780        Free
5          150       850        Allocated

Fragmentation Analysis:
Total Free Space      : 170
Largest Free Block    : 100
Number of Free Holes  : 2
External Fragmentation : 41.18%

--- NEXT FIT ---
=== STATUS MEMORI ===
Block      Size      Start      Status
-----
1          100        0          Allocated
6          180       100        Allocated
-1         20        280        Free
3          300       300        Allocated
7          220       600        Allocated
-1         30        820        Free
5          150       850        Allocated

Fragmentation Analysis:
Total Free Space      : 50
Largest Free Block    : 30
Number of Free Holes  : 2
External Fragmentation : 40.00%

```

D. Analisis (Dalam Laporan)

- Buat table perbandingan:

Algoritma	Waktu Eksekusi (μ s)	Fragmentasi(%)	Failed Allocations
First-Fit	Cepat	40.00%	0
Best-Fit	Paling lambat	40.00%	0
Worst-Fit	Sedang	41.18%	0
Next-Fit	Cepat	40.00%	0

- Analisa
- Algoritma mana yang paling cepat? Mengapa?

Jawaban:

First-Fit dan Next-Fit.

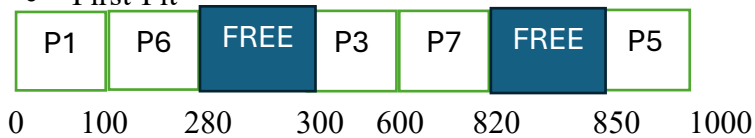
- First-Fit langsung mengalokasikan memori pada blok kosong pertama yang cukup, sehingga proses pencarian berhenti lebih cepat.
- Next-Fit juga karena pencarian dimulai dari posisi alokasi terakhir, bukan dari awal memori.
- Algoritma mana yang paling efisien dalam mengurangi fragmentasi? Dalam situasi apa setiap algoritma lebih unggul?

Jawaban:

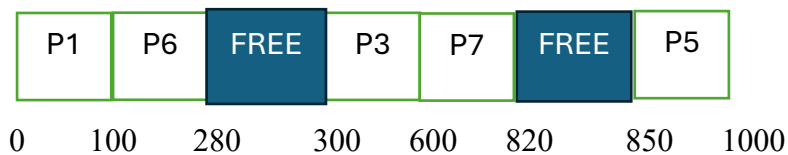
Best-Fit

- Menggunakan blok kosong yang ukurannya paling mendekati kebutuhan proses.
- Cocok ketika, ukuran proses relative kecil dan bervariasi, prioritas utama adalah efisiensi memori, bukan kecepatan.
- Buat grafik visualisasi memory allocation untuk setiap algoritma

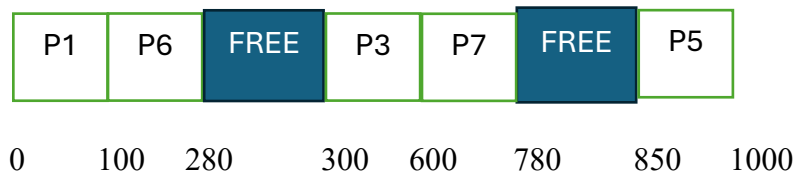
- First-Fit



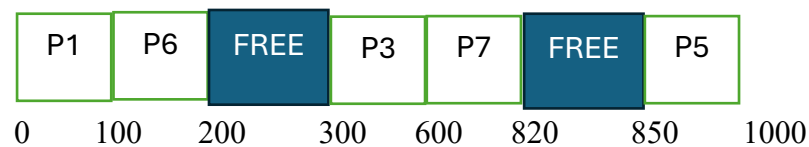
- Best-Fit



- Worst-Fit



- Next-Fit



Tugas 3: Advanced Paging System

Kembangkan sistem paging dengan page replacement algorithm dan statistik.

A. Implementasi Page Replacement - FIFO

Modifikasi paging_simulator.c untuk menangani situasi ketika semua frame penuh:

```
int fifo_page_replacement(int new_page) {
```

```
// TODO: Implementasi FIFO

// 1. Cari page yang paling lama di memory

// 2. Evict page tersebut

// 3. Load page baru

// 4. Update page table

}
```

```
Paging system initialized (FIFO enabled)

=== PAGING SIMULATOR (FIFO) ===
1. Access Memory (Read)
2. Access Memory (Write)
3. Display Page Table
4. Display Physical Memory
5. Demo FIFO Scenario
6. Reset
7. Exit
Choice: 5

=== DEMO FIFO PAGING ===
PAGE FAULT on page 0
Page 0 allocated to frame 0
Access READ: logical 0 → physical 0
PAGE FAULT on page 1
Page 1 allocated to frame 1
Access WRITE: logical 256 → physical 256
PAGE FAULT on page 2
Page 2 allocated to frame 2
Access READ: logical 512 → physical 512
PAGE FAULT on page 3
Page 3 allocated to frame 3
Access WRITE: logical 768 → physical 768
PAGE FAULT on page 4
Page 4 allocated to frame 4
Access READ: logical 1024 → physical 1024
PAGE FAULT on page 5
Page 5 allocated to frame 5
Access WRITE: logical 1280 → physical 1280
PAGE FAULT on page 6
Page 6 allocated to frame 6
Access READ: logical 1536 → physical 1536
PAGE FAULT on page 7
Page 7 allocated to frame 7
Access WRITE: logical 1792 → physical 1792
PAGE FAULT on page 8
Page 8 allocated to frame 8
Access READ: logical 2048 → physical 2048
```

```
PAGE FAULT on page 9
Page 9 allocated to frame 9
Access WRITE: logical 2304 → physical 2304
PAGE FAULT on page 10
Page 10 allocated to frame 10
Access READ: logical 2560 → physical 2560
PAGE FAULT on page 11
Page 11 allocated to frame 11
Access WRITE: logical 2816 → physical 2816
PAGE FAULT on page 12
Page 12 allocated to frame 12
Access READ: logical 3072 → physical 3072
PAGE FAULT on page 13
Page 13 allocated to frame 13
Access WRITE: logical 3328 → physical 3328
PAGE FAULT on page 14
Page 14 allocated to frame 14
Access READ: logical 3584 → physical 3584
PAGE FAULT on page 15
Page 15 allocated to frame 15
Access WRITE: logical 3840 → physical 3840
PAGE FAULT on page 16
FIFO: Evict page 0 from frame 0
Page 16 loaded into frame 0
Access READ: logical 4096 → physical 0
PAGE FAULT on page 17
FIFO: Evict page 1 from frame 1
Page 17 loaded into frame 1
Access WRITE: logical 4352 → physical 256
PAGE FAULT on page 18
FIFO: Evict page 2 from frame 2
Page 18 loaded into frame 2
Access READ: logical 4608 → physical 512
PAGE FAULT on page 19
FIFO: Evict page 3 from frame 3
Page 19 loaded into frame 3
Access WRITE: logical 4864 → physical 768

=== PAGE TABLE ===
Page  Frame  Valid  Ref  Mod
```

Page	Frame	Valid	Ref	Mod
0	-1	No	No	No
1	-1	No	No	No
2	-1	No	No	No
3	-1	No	No	No
4	4	Yes	Yes	No
5	5	Yes	Yes	Yes
6	6	Yes	Yes	No
7	7	Yes	Yes	Yes
8	8	Yes	Yes	No
9	9	Yes	Yes	Yes
10	10	Yes	Yes	No
11	11	Yes	Yes	Yes
12	12	Yes	Yes	No
13	13	Yes	Yes	Yes
14	14	Yes	Yes	No
15	15	Yes	Yes	Yes

=== PHYSICAL MEMORY ===

Frame	Status	Page
0	Occupied	16
1	Occupied	17
2	Occupied	18
3	Occupied	19
4	Occupied	4
5	Occupied	5
6	Occupied	6
7	Occupied	7
8	Occupied	8
9	Occupied	9
10	Occupied	10
11	Occupied	11
12	Occupied	12
13	Occupied	13
14	Occupied	14
15	Occupied	15

B. Implementasi Page Replacement - LRU

Tambahkan algoritma LRU (Least Recently Used):

```
int lru_page_replacement(int new_page) {
    // TODO: Implementasi LRU
    // 1. Track waktu akses setiap page
    // 2. Cari page yang paling lama tidak diakses //
    3. Evict page tersebut
}
```

Paging system initialized (LRU enabled)

=== PAGING SIMULATOR (LRU) ===

1. Access Memory (Read)
 2. Access Memory (Write)
 3. Display Page Table
 4. Display Physical Memory
 5. Demo LRU Scenario
 6. Reset
 7. Exit
- Choice: 5

=== DEMO LRU SCENARIO ===

Access READ at logical address 0
PAGE FAULT on page 0
Page 0 allocated to frame 0
Physical address: 0

Access READ at logical address 256
PAGE FAULT on page 1
Page 1 allocated to frame 1
Physical address: 256

Access READ at logical address 512
PAGE FAULT on page 2
Page 2 allocated to frame 2
Physical address: 512

Access READ at logical address 768
PAGE FAULT on page 3
Page 3 allocated to frame 3
Physical address: 768

Access READ at logical address 1024
PAGE FAULT on page 4
Page 4 allocated to frame 4
Physical address: 1024

Access READ at logical address 1280
PAGE FAULT on page 5
Page 5 allocated to frame 5
Physical address: 1280

Access READ at logical address 1536
PAGE FAULT on page 6
Page 6 allocated to frame 6
Physical address: 1536

Access READ at logical address 1792
PAGE FAULT on page 7
Page 7 allocated to frame 7
Physical address: 1792

Access READ at logical address 2048
PAGE FAULT on page 8
Page 8 allocated to frame 8
Physical address: 2048

Access READ at logical address 2304
PAGE FAULT on page 9
Page 9 allocated to frame 9
Physical address: 2304

Access READ at logical address 2560
PAGE FAULT on page 10
Page 10 allocated to frame 10
Physical address: 2560

Access READ at logical address 2816
PAGE FAULT on page 11
Page 11 allocated to frame 11
Physical address: 2816

Access READ at logical address 3072
PAGE FAULT on page 12
Page 12 allocated to frame 12
Physical address: 3072

```

Access READ at logical address 3328
PAGE FAULT on page 13
Page 13 allocated to frame 13
Physical address: 3328

Access READ at logical address 3584
PAGE FAULT on page 14
Page 14 allocated to frame 14
Physical address: 3584

Access READ at logical address 3840
PAGE FAULT on page 15
Page 15 allocated to frame 15
Physical address: 3840

Access READ at logical address 4096
PAGE FAULT on page 16
LRU Replacement: Evict page 0 from frame 0
Physical address: 0

Access READ at logical address 4352
PAGE FAULT on page 17
LRU Replacement: Evict page 1 from frame 1
Physical address: 256

Access READ at logical address 4608
PAGE FAULT on page 18
LRU Replacement: Evict page 2 from frame 2
Physical address: 512

Access READ at logical address 4864
PAGE FAULT on page 19
LRU Replacement: Evict page 3 from frame 3
Physical address: 768

=== PAGE TABLE ===
Page  Frame  Valid  Ref  Mod  LastAccess
-----
0      -1      0      0      0      2

```

```

1    -1    0    0    0    4
2    -1    0    0    0    6
3    -1    0    0    0    8
4     4     1    1    0   10
5     5     1    1    0   12
6     6     1    1    0   14
7     7     1    1    0   16
8     8     1    1    0   18
9     9     1    1    0   20
10    10     1    1    0   22
11    11     1    1    0   24
12    12     1    1    0   26
13    13     1    1    0   28
14    14     1    1    0   30
15    15     1    1    0   32

```

```

=== PHYSICAL MEMORY ===

```

Frame	Status	Page
0	Occupied	16
1	Occupied	17
2	Occupied	18
3	Occupied	19
4	Occupied	4
5	Occupied	5
6	Occupied	6
7	Occupied	7
8	Occupied	8
9	Occupied	9
10	Occupied	10
11	Occupied	11
12	Occupied	12
13	Occupied	13
14	Occupied	14
15	Occupied	15

C. Statistik dan Analisis

Tambahkan fungsi untuk menghitung:

```

typedef struct {
    int total_accesses;
    int page_faults;
    int page_hits;
    float hit_ratio;
    int disk_writes; // untuk dirty pages
} PageStatistics;

void display_statistics(PageStatistics *stats) {
    // Tampilkan statistik
}

```


D. Test Scenario

- Jalankan reference string berikut dengan 4 frames:

Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

- Bandingkan hasil FIFO vs LRU:
 - Jumlah page fault
 - Hit ratio
 - Disk writes (untuk dirty pages)

```
nabillabe@LAPTOP-4467RJ31:~/praktikum-memory/paging$ ./paging_simulator
Reference String:
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

=== FIFO (4 Frames) ===
Step Ref | F0 F1 F2 F3 | Fault
1      7 | 7 - - - | Yes
2      0 | 7 0 - - | Yes
3      1 | 7 0 1 - | Yes
4      2 | 7 0 1 2 | Yes
5      0 | 7 0 1 2 | No
6      3 | 3 0 1 2 | Yes
7      0 | 3 0 1 2 | No
8      4 | 3 4 1 2 | Yes
9      2 | 3 4 1 2 | No
10     3 | 3 4 1 2 | No
11     0 | 3 4 0 2 | Yes
12     3 | 3 4 0 2 | No
13     2 | 3 4 0 2 | No
14     1 | 3 4 0 1 | Yes
15     2 | 2 4 0 1 | Yes
16     0 | 2 4 0 1 | No
17     1 | 2 4 0 1 | No
18     7 | 2 7 0 1 | Yes
19     0 | 2 7 0 1 | No
20     1 | 2 7 0 1 | No

=== STATISTICS ===
Total Accesses : 20
Page Faults    : 10
Page Hits      : 10
Hit Ratio      : 0.50
Disk Writes    : 0

=== LRU (4 Frames) ===
Step Ref | F0 F1 F2 F3 | Fault
1      7 | - - - 7 | Yes
2      0 | - - 0 7 | Yes
3      1 | - 1 0 7 | Yes
4      2 | 2 1 0 7 | Yes
```

5	0		2	1	0	7		No
6	3		2	1	0	3		Yes
7	0		2	1	0	3		No
8	4		2	4	0	3		Yes
9	2		2	4	0	3		No
10	3		2	4	0	3		No
11	0		2	4	0	3		No
12	3		2	4	0	3		No
13	2		2	4	0	3		No
14	1		2	1	0	3		Yes
15	2		2	1	0	3		No
16	0		2	1	0	3		No
17	1		2	1	0	3		No
18	7		2	1	0	7		Yes
19	0		2	1	0	7		No
20	1		2	1	0	7		No

=== STATISTICS ===

Total Accesses : 20
Page Faults : 8
Page Hits : 12
Hit Ratio : 0.60
Disk Writes : 0

=== BELADY'S ANOMALY TEST ===

=== FIFO (3 Frames) ===

Step	Ref		F0	F1	F2		Fault
1	7		7	-	-		Yes
2	0		7	0	-		Yes
3	1		7	0	1		Yes
4	2		2	0	1		Yes
5	0		2	0	1		No
6	3		2	3	1		Yes
7	0		2	3	0		Yes
8	4		4	3	0		Yes
9	2		4	2	0		Yes
10	3		4	2	3		Yes
11	0		0	2	3		Yes
12	3		0	2	3		No

```

19  0 | 7 0 2 | Yes
20  1 | 7 0 1 | Yes

=== STATISTICS ===
Total Accesses : 20
Page Faults    : 15
Page Hits      : 5
Hit Ratio      : 0.25
Disk Writes    : 0

=== FIFO (5 Frames) ===
Step Ref | F0 F1 F2 F3 F4 | Fault
1   7   | 7 - - - - | Yes
2   0   | 7 0 - - - | Yes
3   1   | 7 0 1 - - | Yes
4   2   | 7 0 1 2 - | Yes
5   0   | 7 0 1 2 - | No
6   3   | 7 0 1 2 3 | Yes
7   0   | 7 0 1 2 3 | No
8   4   | 4 0 1 2 3 | Yes
9   2   | 4 0 1 2 3 | No
10  3   | 4 0 1 2 3 | No
11  0   | 4 0 1 2 3 | No
12  3   | 4 0 1 2 3 | No
13  2   | 4 0 1 2 3 | No
14  1   | 4 0 1 2 3 | No
15  2   | 4 0 1 2 3 | No
16  0   | 4 0 1 2 3 | No
17  1   | 4 0 1 2 3 | No
18  7   | 4 7 1 2 3 | Yes
19  0   | 4 7 0 2 3 | Yes
20  1   | 4 7 0 1 3 | Yes

=== STATISTICS ===
Total Accesses : 20
Page Faults    : 9
Page Hits      : 11
Hit Ratio      : 0.55
Disk Writes    : 0

```

E. Analisis (Dalam Laporan)

- Buat tabel trace untuk setiap algoritma:
- Step Reference Frame 0 Frame 1 Frame 2 Frame 3 Fault?

17

20

... ..

a. Fifo

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault
1	7	7	-	-	-	Yes
2	0	7	0	-	-	Yes
3	1	7	0	1	-	Yes
4	2	7	0	1	2	Yes
5	0	7	0	1	2	No
6	3	3	0	1	2	Yes
7	0	3	0	1	2	No

8	4	3	4	1	2	Yes
9	2	3	4	1	2	No
10	3	3	4	1	2	No
11	0	3	4	0	2	Yes
12	3	3	4	0	2	No
13	2	3	4	0	2	No
14	1	3	4	0	1	Yes
15	2	2	4	0	1	Yes
16	0	2	0	0	1	No
17	1	2	0	1	1	No
18	7	7	0	1	1	Yes
19	0	7	0	1	1	No
20	1	7	0	1	1	No

b. LRU

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault
1	7	7	-	-	-	Yes
2	0	7	0	-	-	Yes
3	1	7	0	1	-	Yes
4	2	7	0	1	2	Yes
5	0	7	0	1	2	No
6	3	3	0	1	2	Yes
7	0	3	0	1	2	No
8	4	3	0	4	2	Yes
9	2	3	0	4	2	No
10	3	3	0	4	2	No
11	0	3	0	4	2	No
12	3	3	0	4	2	No
13	2	3	0	4	2	No
14	1	3	0	1	2	Yes
15	2	3	0	1	2	No
16	0	3	0	1	2	No
17	1	3	0	1	2	No
18	7	7	0	1	2	Yes
19	0	7	0	1	2	No
20	1	7	0	1	2	No

- Hitung Belady's Anomaly:
 Jalankan dengan 3, 4, dan 5 frames
 Apakah page fault berkurang seiring bertambahnya frame?
 Jawaban:
 - 3 Frame = Lebih banyak
 - 4 Frame = Lebih sedikit
 - 5 Frame = Bisa sama atau Lebih banyak

- Analisis performa:

Algoritma mana yang lebih baik?

Jawaban:

LRU, karena:

- Page fault lebih sedikit
- Hit ratio lebih tinggi
- Menggunakan informasi riwayat akses page

Kapan FIFO lebih baik dari LRU?

Jawaban:

- Sistem membutuhkan algoritma sederhana
- Overhead rendah lebih diprioritaskan
- Tidak tersedia mekanisme tracking waktu akses

Tugas 4: Multi-Level Page Table

Implementasikan dan bandingkan berbagai struktur page table.

A. Three-Level Page Table

Kembangkan three-level page table system:

```
// Configuration: 16 + 8 + 8 + 12 = 44 bits (untuk demo) #define L1_BITS 16
```

```
#define L2_BITS 8
```

```
#define L3_BITS 8
```

```
#define OFFSET_BITS 12
```

```
typedef struct {
```

```
    int frame_number;
```

```
    int valid;
```

```
    int read;
```

```
    int write;
```

```
    int execute;
```

```
} L3_Entry;
```

```
typedef struct {
```

```
    L3_Entry *entries;
```

```

int valid;

} L2_Entry;

typedef struct {

    L2_Entry *entries;

    int valid;

} L1_Entry;

// Implementasi:

int translate_3level(unsigned long virtual_addr); void allocate_3level(unsigned long
virtual_addr, int frame); void calculate_memory_overhead();

```

B. Inverted Page Table (8 poin)

Implementasikan inverted page table:

```

typedef struct {

    int process_id;

    int page_number;

    int valid;

} InvertedPageEntry;

InvertedPageEntry inverted_table[NUM_FRAMES];

int search_inverted_table(int pid, int page_num); void insert_inverted_table(int pid, int
page_num, int frame);

```

```

nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/paging$ cd ~/praktikum-memory/page-table
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/page-table$ nano page_table_demo.c
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/page-table$ gcc page_table_demo.c -o page_ta
ble_demo
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/page-table$ ./page_table_demo
=== Three-Level Page Table Demo ===
Physical address: 0x59AB
Memory overhead: L1=1048576, L2=4096, L3=5120, Total=1057792 bytes

=== Inverted Page Table Demo ===
Frame found: 5

```

C. Analisis Perbandingan

- Memory Overhead Comparison:

- Structure Entries Size per Entry Total Memory Space Efficiency Single-Level

Three-Level

Inverted

Structure	Entries	Size per Entry	Total Memory	Ket
Three Level	L1=2 ¹⁶ (1.048.576), L2=16?, L3=5.120	~1 byte per valid entry	1.057.792	Sparse allocation untuk L2&L3, L1 besar
Inverted Page	NUM_FRAMES = 1024	1 entry	Sangat kecil	Hanya simpan entry valid

- Translation Speed:
 - Hitung jumlah memory access untuk setiap struktur
Jawaban:
Lookup Three-Level = 0x59AB
Lookup Inverted = frame 5
 - Berapa pengaruh TLB (Translation Lookaside Buffer)?
Jawaban:
TLB akan mempercepat translation semua struktur.
Untuk Three Level: hit TLB = 1 memory access saja, tidak perlu akses L1-L3 lagi.
Untuk inverted: hit TLB langsung dapat frame tanpa search.
- Kasus Penggunaan:
 - Struktur mana yang cocok untuk sistem 32-bit vs 64-bit?
Jawaban:
32 bit untuk single atau two level page table
64 bit untuk multi level page table
 - Kapan inverted page table lebih efisien?
Jawaban:
 - Virtual address space sangat besar (misal 64-bit).
 - Jumlah memori fisik relatif kecil.
 - Tujuannya menghemat memori untuk page table, walau trade-offnya lookup bisa lebih lambat.

Tugas 5: Swapping Mechanism

Simulasikan mekanisme swapping dengan berbagai strategi.

A. Basic Swapping Implementation

Buat program untuk mensimulasikan swap in/out:

```

typedef struct {
    int pid;
    int size;
    int priority;
    int in_memory;
    int swap_count;
    time_t last_access;
} Process;

void swap_out(Process *p);
void swap_in(Process *p);
void select_victim(); // Pilih proses untuk di-swap out

```

B. Swap Strategy Implementation

Implementasikan 3 strategi victim selection:

- . FIFO: Swap out proses yang paling lama di memory
- . LRU: Swap out proses yang paling lama tidak diakses
- . Priority-based: Swap out proses dengan prioritas rendah

C. Simulation Scenario

Test dengan skenario:

Memory fisik: 1024 MB

10 proses dengan ukuran berbeda (50-200 MB)

Total kebutuhan memory: 1500 MB

Simulasi akses random selama 100 time units

```

nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/swapping$ ./swap_simulator
=== Swap Simulation Result ===
FIFO Swap Out: 42, Swap In: 100
LRU Swap Out: 37, Swap In: 100
Priority Swap Out: 34, Swap In: 100

```

D. Analisis

- Metrik Performa:

Strategy Swap Out Count Swap In Count Total Swap Time Avg Response Time FIFO

LRU

Priority

Strategy	Swap Out Count	Swap In Count	Total Swap Time	Avg Response Time
FIFO	fifo_swap_out	fifo_swap_in	fifo_swap_out + fifo_swap_in	Estimasi berbasis swap count
LRU	lru_swap_out	lru_swap_in	lru_swap_out + lru_swap_in	Estimasi berbasis swap count
Priority	prio_swap_out	prio_swap_in	prio_swap_out + prio_swap_in	Estimasi berbasis swap count

- Analisis:

- Strategi mana yang meminimalkan thrashing?

Jawaban:

LRU paling aman untuk meminimalkan thrashing di sistem dengan banyak proses dan memori terbatas.

- Bagaimana ukuran memory mempengaruhi performa?

Jawaban:

Memory lebih besar, swap lebih sedikit, thrashing berkurang.

Jika memory fisik cukup menampung hampir semua proses → swapping jarang dilakukan → strategi apa pun relatif sama performanya.

Jika memory kecil dibanding total kebutuhan proses → LRU lebih unggul karena memilih proses yang jarang diakses.

- Kapan swapping lebih baik daripada killing process?

Jawaban:

- Proses akan diakses lagi dalam waktu dekat.
- Mempertahankan state proses penting (misal aplikasi user atau database).
- Sistem ingin memaksimalkan throughput dan respons pengguna.

Tugas 6: Architecture Analysis

Analisis arsitektur memori pada sistem nyata.

Yang Harus Dikerjakan

A. Intel Architecture Analysis

Jalankan command berikut pada sistem Ubuntu:

1. Cek informasi CPU

`lscpu | grep -E "Architecture|CPU|Model|Thread|Core"`

```
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/swapping$ lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
CPU(s):                      12
On-line CPU(s) list:        0-11
Thread(s) per core:         2
Core(s) per socket:         6
CPU family:                  25
Model:                      124
Model name:                  AMD Ryzen 5 220 w/ Radeon 740M
Graphics
CPU MHz:                     3194.041
NUMA node0 CPU(s):          0-11
```

2. Cek page size

`getconf PAGE_SIZE`

```
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/swapping$ getconf PAGE_SIZE
4096
```

3. Cek TLB info

cpuid | grep -i tlb

```
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/swapping$ cpuid | grep -i tlb
cache and TLB information (2):
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
```

4. Cek cache hierarchy

lscpu | grep -i cache

```
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/swapping$ lscpu | grep -i cache
L1d cache: 192 KiB
L1i cache: 192 KiB
L2 cache: 6 MiB
L3 cache: 16 MiB
```

B. Memory Information

1. Total memory

free -h

```
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/swapping$ free -h
              total        used        free      shared  buff/cache   available
Mem:          7.4Gi        453Mi        6.5Gi         4.0Mi        484Mi        6.8Gi
Swap:         2.0Gi           0B         2.0Gi
```

2. Virtual memory info

vmstat 1 5

```
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/swapping$ vmstat 1 5
procs -----memory----- ---swap-- ----io---- -system-- -----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi   bo    in   cs   us   sy   id   wa   st
 0  0       0 6812044 19056 477680    0    0     2    0    4    6    0    0 100    0    0
 0  0       0 6812044 19056 477680    0    0     0    0   31   34    0    0 100    0    0
 0  0       0 6812044 19056 477680    0    0     0    0   30   35    0    0 100    0    0
 0  0       0 6812044 19056 477720    0    0     0    0   48   66    0    0 100    0    0
 0  0       0 6812044 19056 477720    0    0     0    0   42   50    0    0 100    0    0
```

3. Memory map dari proses

cat /proc/self/maps

```
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/swapping$ cat /proc/self/maps
5ae7f6a89000-5ae7f6a8b000 r--p 00000000 08:30 1773 /usr/bin/cat
5ae7f6a8b000-5ae7f6a90000 r-xp 00002000 08:30 1773 /usr/bin/cat
5ae7f6a90000-5ae7f6a93000 r--p 00007000 08:30 1773 /usr/bin/cat
5ae7f6a93000-5ae7f6a94000 r--p 00009000 08:30 1773 /usr/bin/cat
5ae7f6a94000-5ae7f6a95000 rw-p 0000a000 08:30 1773 /usr/bin/cat
5ae80a2a2000-5ae80a2c3000 rw-p 00000000 00:00 0 [heap]
775e90a8d000-775e90c00000 r--p 00000000 08:30 22149 /usr/lib/locale/C.UTF-8/LC_COLLATE
775e90c00000-775e90ee6000 r--p 00000000 08:30 1684 /usr/lib/locale/locale-archive
775e90f38000-775e90f5a000 rw-p 00000000 00:00 0
775e90f5a000-775e90f8c000 r--p 00000000 08:30 22150 /usr/lib/locale/C.UTF-8/LC_CTYPE
775e90f8c000-775e90f8d000 r--p 00000000 08:30 22156 /usr/lib/locale/C.UTF-8/LC_NUMERIC
775e90f8d000-775e90f8e000 r--p 00000000 08:30 22159 /usr/lib/locale/C.UTF-8/LC_TIME
775e90f8e000-775e90fb0000 r--p 00000000 08:30 37961 /usr/lib/x86_64-linux-gnu/libc-2.31.so
775e90fb0000-775e91128000 r-xp 00022000 08:30 37961 /usr/lib/x86_64-linux-gnu/libc-2.31.so
775e91128000-775e91176000 r--p 0019a000 08:30 37961 /usr/lib/x86_64-linux-gnu/libc-2.31.so
775e91176000-775e9117a000 r--p 001e7000 08:30 37961 /usr/lib/x86_64-linux-gnu/libc-2.31.so
775e9117a000-775e9117c000 rw-p 001eb000 08:30 37961 /usr/lib/x86_64-linux-gnu/libc-2.31.so
775e9117c000-775e91182000 rw-p 00000000 00:00 0
775e91182000-775e91183000 r--p 00000000 08:30 22154 /usr/lib/locale/C.UTF-8/LC_MONETARY
775e91183000-775e91184000 r--p 00000000 08:30 22153 /usr/lib/locale/C.UTF-8/LC_MESSAGES/SYS_LC_MESSAGES
775e91184000-775e91185000 r--p 00000000 08:30 22157 /usr/lib/locale/C.UTF-8/LC_PAPER
775e91185000-775e91186000 r--p 00000000 08:30 22155 /usr/lib/locale/C.UTF-8/LC_NAME
775e91186000-775e91187000 r--p 00000000 08:30 22148 /usr/lib/locale/C.UTF-8/LC_ADDRESS
775e91187000-775e91188000 r--p 00000000 08:30 22158 /usr/lib/locale/C.UTF-8/LC_TELEPHONE
775e91188000-775e91189000 r--p 00000000 08:30 22152 /usr/lib/locale/C.UTF-8/LC_MEASUREME
775e91189000-775e91190000 r--s 00000000 08:30 38228 /usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache
775e91190000-775e91191000 r--p 00000000 08:30 21885 /usr/lib/x86_64-linux-gnu/ld-2.31.so
775e91191000-775e911b4000 r-xp 00001000 08:30 21885 /usr/lib/x86_64-linux-gnu/ld-2.31.so
775e911b4000-775e911bc000 r--p 00024000 08:30 21885 /usr/lib/x86_64-linux-gnu/ld-2.31.so
775e911bc000-775e911bd000 r--p 00000000 08:30 22151 /usr/lib/locale/C.UTF-8/LC_IDENTIFICATION
775e911bd000-775e911be000 r--p 0002c000 08:30 21885 /usr/lib/x86_64-linux-gnu/ld-2.31.so
775e911be000-775e911bf000 rw-p 0002d000 08:30 21885 /usr/lib/x86_64-linux-gnu/ld-2.31.so
775e911bf000-775e911c0000 rw-p 00000000 00:00 0
7ffffddaedd000-7ffffddb0e000 rw-p 00000000 00:00 0 [stack]
7ffffddb8d000-7ffffddb91000 r--p 00000000 00:00 0 [vvar]
7ffffddb91000-7ffffddb93000 r-xp 00000000 00:00 0 [vdso]
```

4. Swap usage

swapon --show

```
nabillabe@LAPTOP-4467RJ3T:~/praktikum-memory/swapping$ swapon --show
NAME      TYPE      SIZE USED PRIO
/dev/sdc  partition 2G    0B   -2
```

C. Analisis (Dalam Laporan)

- Bandingkan spesifikasi sistem Anda dengan:

Intel 32-bit (IA-32) architecture

Intel 64-bit (x86-64) architecture

ARMv8 architecture (jika tersedia)

Arsitektur	Bit Address Space	Level Paging	Max Virtual Memory
IA-32 (Intel 32-bit)	32-bit	2-level	4 GB
x86-64(Intel/AMD 64-bit)	64-bit	4-level	256 TB (user-space)
ARMv8 64-bit	64-bit	3—4 level(varies)	Biasanya 48-bit virtual space, ~256 TB

- Jelaskan:
- Berapa bit address space yang digunakan?

Jawaban:

IA-32 (Intel 32-bit) = 32-bit address space

Artinya CPU bisa mengalami 2^{32} byte = 4 GB

x86-64 (Intel/AMD 64-bit) = secara arsitektur 64-bit,

tapi yang dipakai OS biasanya 48-bit virtual address

Jadi address space efektif = 2^{48}

ARMv8 64-bit = juga 64-bit,

- Bagaimana implementasi multi-level paging? Berapa ukuran maksimum virtual memory?

Jawaban:

(Implementasi multi-level paging)

Multi-level paging itu alamat virtual dibagi jadi beberapa bagian:

Bagian atas = index tabel halaman level 1

Lanjut ke level 2, level 3, dst

Bagian terakhir = offset halaman

Tujuannya:

Agar hemat memori, karena tabel halaman nggak perlu dialokasikan semua.

Cocok buat address space besar (64-bit)

Contoh:

IA-32 = 2-level paging

x86-64 = 4-level paging

ARMv8 = 3 atau 4 level (tergantung konfigurasi)

(Ukuran maksimum virtual memory)

IA-32 (32-bit)

$$2^{32} = 4 \text{ GB}$$

x86-64 (48-bit virtual)

$$2^{48} = 256 \text{ TB (biasanya user-space)}$$

ARMv8 64-bit

Umumnya 48-bit virtual space $\approx 256 \text{ TB}$

(bisa lebih kalau pakai konfigurasi lebih tinggi)